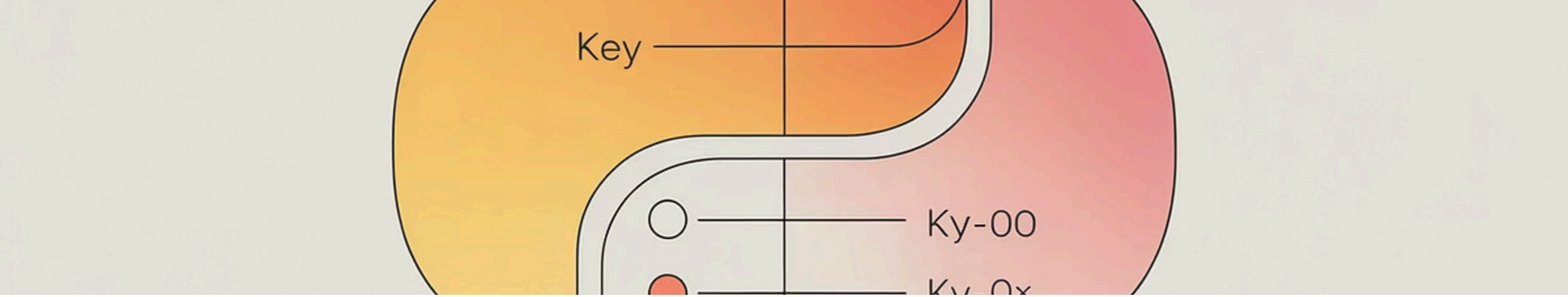
Lección 3: Diccionarios y Sets

Estructuras No Ordenadas

Descubre cómo almacenar y manipular datos sin un orden fijo, utilizando claves para acceder a información y operaciones lógicas de conjuntos para análisis avanzado.

Made with GAMMA



Diccionarios (dict): Acceso por Clave

¿Qué es un diccionario?

Los diccionarios se definen con **llaves** ({}) y almacenan pares de **clave: valor**.

```
perfil = {"nombre": "Juan",
          "edad": 30,
          "activo": True}

print(perfil["edad"])
# Resultado: 30
```

Características principales

- **Mutable:** Puedes modificar, añadir o eliminar elementos
- **Acceso directo:** Recupera valores mediante sus claves
- **Orden de inserción:** Python mantiene el orden (versiones 3.7+)

Enfoque para Análisis de Datos: Los diccionarios simulan registros de bases de datos o documentos JSON, mapeando identificadores (claves) a su información (valores).

Made with GAMMA

Diccionarios: Manipulación Básica

Agregar, Actualizar y Eliminar Datos

Añadir elementos

Usa corchetes con una nueva clave para crear un elemento:

```
perfil["ciudad"] = "Madrid"
```

Actualizar valores

Si la clave existe, se actualiza su valor:

```
perfil["edad"] = 31
```

Eliminar elementos

Utiliza la palabra clave **del** para eliminar:

```
del perfil["activo"]
```

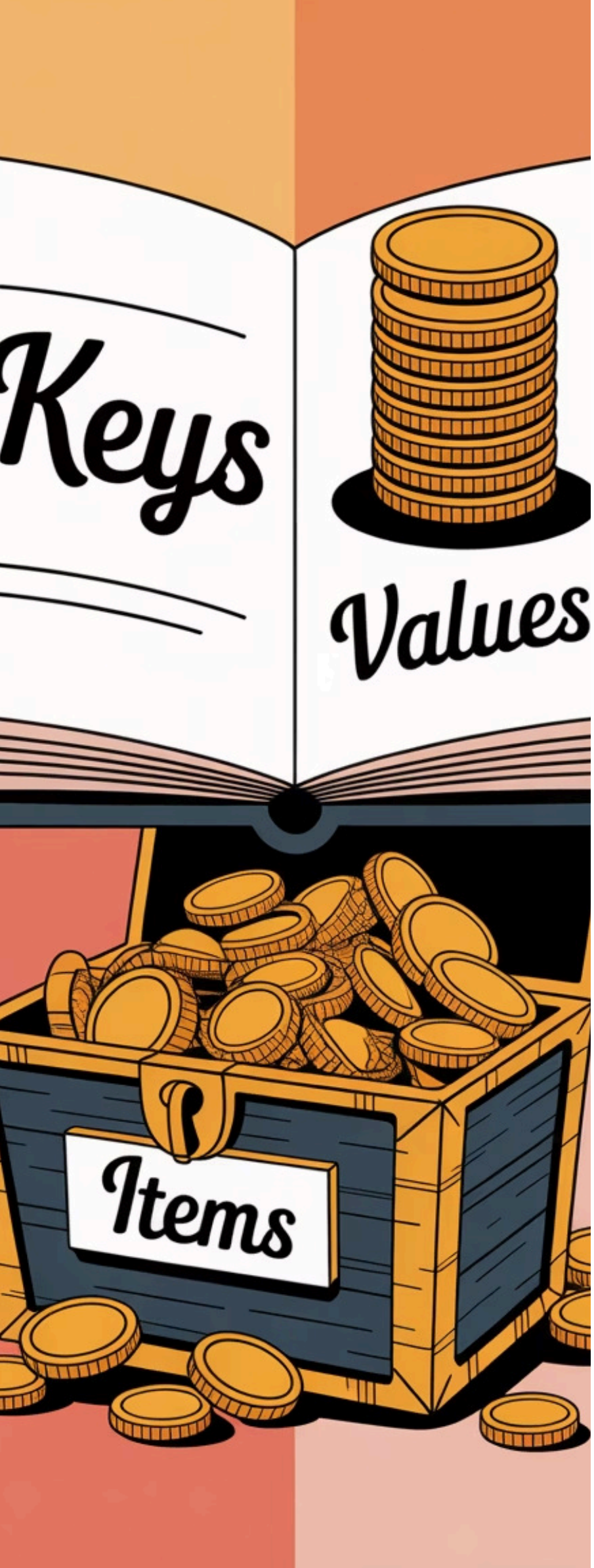
Acceso seguro con .get()

El método **.get()** te permite acceder a valores sin riesgo de errores. Si la clave no existe, devuelve **None** o un valor por defecto que especifiques.

```
email = perfil.get("email",
                   "No tiene email")
```

Esta práctica es esencial en análisis de datos para manejar campos opcionales o incompletos.

Made with GAMMA



Diccionarios: Métodos

Inspeccionando las Claves y Valores

.keys()

Devuelve una vista iterable de **todas las claves** del diccionario.

```
claves = perfil.keys()
```

Uso en Análisis: Listar los nombres de columnas o campos de un registro.

.values()

Devuelve una vista de **todos los valores** almacenados.

```
valores = perfil.values()
```

Uso en Análisis: Extraer solo los datos para realizar cálculos rápidos.

.items()

Devuelve pares **(clave, valor)**. Ideal para iterar en bucles.

```
for k, v in perfil.items():
    print(f"{k}: {v}")
```

Uso en Análisis: Procesar registros completos con sus identificadores.

Tip importante: Para convertir estas vistas en listas o sets, usa los constructores **list()** o **set()**. Ejemplo: list(perfil.keys())

Made with GAMMA

Sets (set):

¿Qué es un set?

Los sets se definen con **llaves** ({}) o usando **set()**. Su superpoder es eliminar automáticamente elementos duplicados.

```
datos = {1, 2, 3, 3, 4}
print(datos)
# Resultado: {1, 2, 3, 4}
```

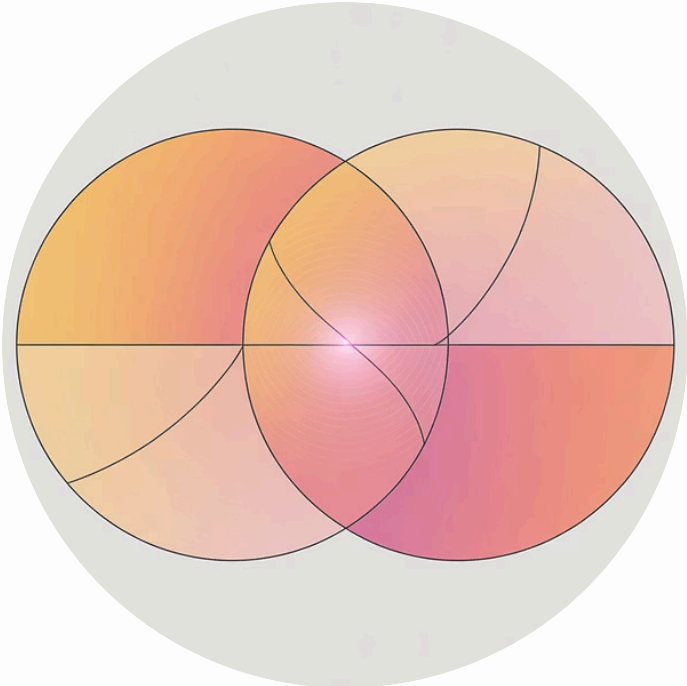
Características principales

- **No ordenado:** No mantiene orden de inserción
- **No indexable:** No puedes usar `set[0]`
- **Solo elementos únicos:** Elimina duplicados automáticamente
- **Mutable:** Puedes añadir y eliminar elementos

Enfoque para Análisis de Datos: Los sets son perfectos para la **limpieza de datos** y la verificación de valores únicos en columnas o conjuntos de información.

Made with GAMMA

Sets

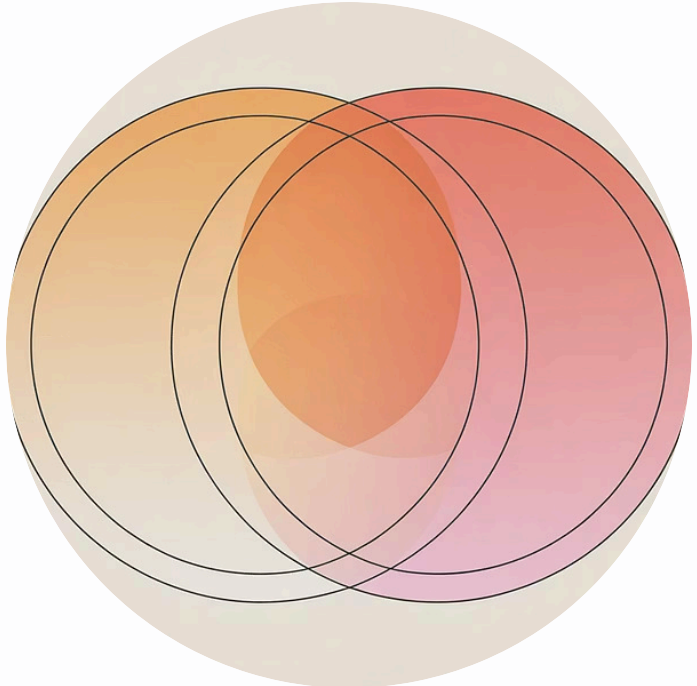


Unión (union o |)

Combina todos los elementos de ambos sets, eliminando duplicados automáticamente.

```
A | B
```

Uso DA: Consolidar dos listas de clientes sin repetir registros.

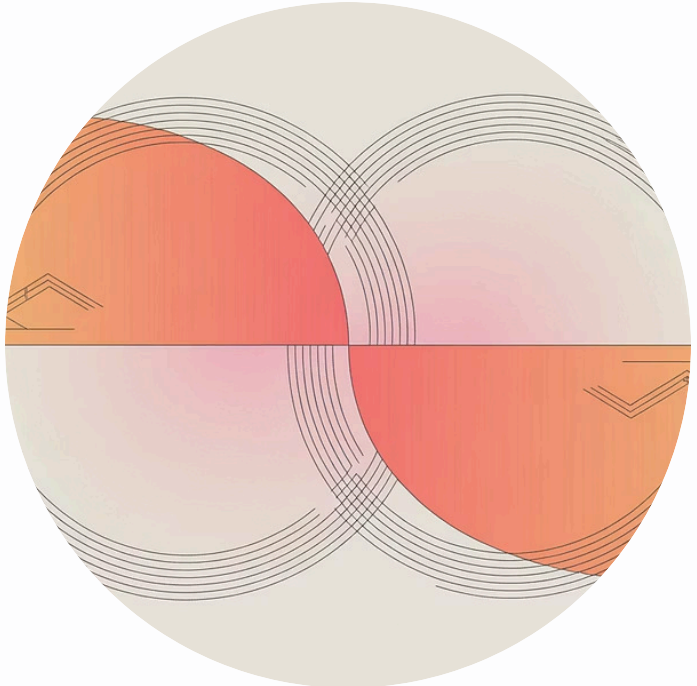


Intersección (intersection o &)

Devuelve solo los elementos que están presentes en **ambos sets**.

```
A & B
```

Uso DA: Encontrar clientes comunes entre dos campañas de marketing.



Diferencia (difference o -)

Devuelve elementos del primer set que **no están** en el segundo.

```
A - B
```

Uso DA: Identificar clientes de Campaña A que NO compraron en Campaña B.

Añadir elementos

```
mi_set.add(elemento)
```

Agrega un nuevo elemento único al set.

Eliminar elementos

```
mi_set.remove(elemento)
```

Elimina un elemento específico del set.

Made with GAMMA

Resumen: Cuatro Estructuras de Datos

¿Cuándo Usar Cada Contenedor?

Estructura	Símbolo	Ordenado	Mutable	Enfoque para Análisis de Datos
Listas	[ ]	Sí (Indexable)	Sí	Flujo de trabajo, limpieza, colecciones flexibles
Tuplas	( )	Sí (Indexable)	No	Integridad, datos fijos (coordenadas, claves)
Diccionarios	{key: value}	No (Acceso por Clave)	Sí	Registros (JSON), mapeo de datos
Sets	{a, b}	No	Sí	Operaciones lógicas de conjuntos

Dominio completo

Has aprendido las cuatro estructuras fundamentales de Python para análisis de datos

Próximos pasos

El siguiente módulo aborda **Lógica de Control** (if, for, while) para manipular estas estructuras automáticamente

¡A practicar!

Aplica estos conceptos en tus proyectos de análisis de datos reales

Made with GAMMA